3-Microphone Delay Estimation Test Set (simulated)

Test data for time-delay estimation / delay-and-sum beamforming with a small 3-microphone triangle array. Generated with `pyroomacoustics` (image-source method) from a real speech recording plus added noise.

Each folder is **one test case:** 3 channels × 1024 samples, plus the correct answer. The format is the same as the Task 0 synthetic test set.

Quick facts

Item	Value
Sample rate	16 000 Hz
Samples per channel	exactly 1024 (64 ms)
Sample format	signed 16-bit, two's complement, stored as hex
Channels	3 (`ch1` , `ch2` , `ch3`)
Source	speech recording, played from a point source 1.5 m away
Total cases	432 (36 angles × 3 echo levels × 4 noise levels × 1 window)

Folder layout

```
task5_dataset/
├── index.csv                                  <- one row per case, easy to load
├── rt60_0.00_snr_30_angle_000_w1/
│   ├── ch1.mem
│   ├── ch2.mem
│   ├── ch3.mem
│   ├── expected.json
│   └── meta.json
├── rt60_0.00_snr_30_angle_010_w1/
└── ...
```

Folder name

`rt60_<RT60>_snr_<SNR>_angle_<ANGLE>_w<WINDOW>`

Part	Meaning	Values
`rt60`	Echo level: reverberation time in seconds. `0.00` = anechoic (direct path only, no echoes)	0.00, 0.25, 0.50
`snr`	Signal-to-noise ratio in dB (higher = cleaner)	30, 20, 10, 00
`angle`	Source direction in degrees	000, 010, ... 350
`w`	Window number (which 1024-sample slice of the recording)	1

Example: `rt60_0.25_snr_10_angle_090_w1` = moderate echo, 10 dB SNR, source at 90°.

Files inside each folder

`ch1.mem`, **`ch2.mem`**, **`ch3.mem`**

One microphone each. **1024 lines**, one sample per line, as **4 hex digits** (16-bit two's complement, uppercase):

```
FFFF   ->  -1
0001   ->   1
7FFF   ->   32767
8000   ->   -32768
```

Compatible with Verilog `$readmemh`:

verilog

```verilog
reg [15:0] mic1 [0:1023];
initial $readmemh("ch1.mem", mic1);
```

Reading in Python:

python

```python
import numpy as np
v = np.array([int(line, 16) for line in open("ch1.mem")])
v[v >= 32768] -= 65536      # convert to signed
```

All three channels in a folder are scaled by **one common gain** so the loudest sample is about 30000. This keeps the relative level between microphones intact. Because of this, amplitude should **not** be compared between different folders.

`expected.json`

The correct answer. Only the three pairwise lags, in whole samples:

```json
{
    "ch1_ch2": 0,
    "ch1_ch3": -2,
    "ch2_ch3": -2
}
```

Sign convention: `ch1_ch2 = +k` means the signal reaches `ch2` **k samples later** than `ch1`. Negative means `ch2` hears it earlier. Also `ch2_ch3` is approximately `ch1_ch3 - ch1_ch2` (it can differ by 1 because each lag is rounded separately).

`meta.json`

Everything else about the case (not needed by the algorithm, useful for analysis):

Field	Meaning
`source_angle_deg`	Source direction, counter-clockwise from the +x axis, measured at the array centre
`rt60_s`, `snr_db`	Echo and noise level
`window_start_sample`	Where in the full simulated recording this 1024-sample window starts
`fs_hz`, `speed_of_sound_m_s`	16000 and 343
`ism_max_order`	Image-source reflection order used (0 = anechoic)
`source_position_m`	Source x, y, z in metres
`mic_positions_m`	Positions of mic 1, 2, 3 in metres
`true_lags_samples_fractional`	Exact direct-path lags before rounding
`lag_sign`	Sign convention (as above)

`index.csv` (top level)

One row per case: `folder, angle, rt60, snr_db, ch1_ch2, ch1_ch3, ch2_ch3` plus `ch1_ch2_frac, ch1_ch3_frac, ch2_ch3_frac` (the unrounded lags). Use this to loop over the whole dataset and compute accuracy by angle, SNR or RT60.

Setup that produced the data

Microphone triangle (metres, relative to the array centroid, all at the same height):

Pair	Spacing
mic 1 – mic 2	8 cm
mic 1 – mic 3	5 cm
mic 2 – mic 3	6.4 cm

Array centre (centroid) at (3.0, 2.5, 1.5) m.

Room: shoebox, 6 × 5 × 3 m. The source is 1.5 m from the array centre at the same height, so all directions are in the horizontal plane.

Echo levels: RT60 = 0 (anechoic), 0.25 s, 0.5 s.

Noise: independent white Gaussian noise on each microphone. The SNR is measured on the 1024-sample window itself, so the label is accurate for the window you actually get.

Window choice: the loudest 1024-sample stretch of the simulated speech, so a window never lands in silence.

Things to be aware of

1. **Lag range is small.** With 8 cm spacing the largest possible lag is about 3.7 samples at 16 kHz, so every answer is between **−4 and +4**. The ±13 edge-case lags from the Task 0 test set cannot occur here.
2. **Ground truth is rounded geometry.** `expected.json` is the direct-path delay rounded to the nearest sample. Real delays are fractional, so an estimator can legitimately disagree when the true lag is near x.5 (for example −3.495 → expected −3, but −4 is almost equally valid). Use the `_frac` columns in `index.csv` to flag these cases when scoring.
3. **Echoes and noise make it harder, by design.** Even a good estimator is expected to get worse as RT60 rises and SNR falls. That drop is a result, not a bug.
4. **Direct-path truth only.** In echoey rooms, reflections can pull the correlation peak away from the direct-path lag. The answer key does not account for this.
5. **One speaker, one recording.** All cases come from the same speech recording, so there is no variety in voice or content.

Regenerating and checking

```
python make_dataset.py                  # builds task5_dataset/ (needs
speech.wav beside it)
python verify_dataset.py task5_dataset  # checks files, geometry, and runs a
reference estimator
```

`verify_dataset.py` checks the file format, recomputes the lags from the geometry, and runs an independent GCC-PHAT estimator. It prints accuracy by RT60 and SNR so you can see the

expected trend.